TestNG Complete Interview Guide

94 Unique Questions • Clear Explanations • Real Examples • Code Snippets

10 Topics	94 Questions	Code Examples	Interview Ready

SECTION 1 — Core Concepts & Overview

Q1. What is TestNG?

TestNG (Test Next Generation) is a powerful testing framework for Java inspired by JUnit and NUnit. It is widely used with Selenium for automation. It gives testers full control over test execution — ordering, grouping, parallel runs, and detailed reporting — all from a simple XML file.

■ Example:

Imagine you have 100 test cases for an e-commerce site. Without TestNG, running and managing them is messy. With TestNG, you can label tests as 'smoke' or 'regression', run them in the right order, and get an HTML report automatically.

■ Pro Tip:

TestNG is not just for testers — developers use it for unit and integration testing too.

Q2. What are the key features of TestNG?

TestNG provides: (1) Annotations to control test lifecycle, (2) Test prioritization and ordering, (3) Grouping tests into smoke/regression/sanity, (4) Parallel test execution to save time, (5) Data-driven testing using @DataProvider, (6) Automatic HTML/XML report generation, (7) Dependency between tests, (8) Parameterization from XML.

■ Real World: A QA team uses groups to run only 'smoke' tests after every build. Full regression runs only before release. This saves hours of execution time every day.

Q3. Is TestNG only for Software Testers?

No. TestNG is used by both testers AND developers. Developers use it for unit testing and regression checks during development. Testers use it for end-to-end automation with Selenium.

■ Example:

A developer writes a @Test to check if a login method returns the correct user object. A tester writes a @Test to verify the login page works in Chrome, Firefox, and Edge simultaneously.

Q4. What is a TestNG Suite, Test, Class, and Method?

Think of it as a hierarchy: Suite → Test → Class → Method. The Suite is the top-level container (defined in testng.xml). A Test is a logical group of related classes. A Class is a Java file with test methods. A Method is an individual @Test function.

```
testng.xml (Suite) ■■■ <test name="LoginTests"> ← Test block ■■■ <class name="LoginTest"/> ← Java Class ■■■ @Test login() ← Test Method ■■■ @Test logout() ← Test Method
```

Q5. What is the difference between TestNG and JUnit?

JUnit is simpler and best for unit testing individual methods. TestNG is more powerful and built for large automation frameworks. Key differences: TestNG supports parallel execution, grouping, and dependency — JUnit does not natively. TestNG uses testng.xml for suite configuration; JUnit uses annotations and runners.

■ Example:

JUnit: @Test void checkAdd() { assertEquals(5, 2+3); } — simple unit check. TestNG: same test + groups='smoke', priority=1, dependsOnMethods='login' — full automation control.

Q6. Why do we need TestNG in Selenium?

Selenium alone only automates browser actions. It has NO built-in reporting, NO test ordering, NO grouping, and NO parallel execution. TestNG fills all these gaps. Without TestNG, you'd have no way to know which tests passed, failed, or were skipped in a readable format.

■ Real World: Selenium clicks the button. TestNG tells you: 'Test #42 - LoginTest - FAILED in 3.2s - Screenshot attached.'

SECTION 2 — TestNG Annotations

Q7. What are Annotations in TestNG?

Annotations are special @ keywords placed above methods that tell TestNG WHEN to run that method. They control the entire test lifecycle — what runs before tests, during tests, and after tests.

■ Example:

@BeforeMethod tells TestNG: 'Run this method before EVERY test.' It's like pressing RESET before each game level.

```
@BeforeMethod public void openBrowser() { driver = new ChromeDriver(); // runs before each @Test } @Test public void loginTest() { ... } @AfterMethod public void closeBrowser() { driver.quit(); // runs after each @Test }
```

Q8. What is the execution order of all TestNG annotations?

TestNG always follows this strict order. Think of it like Russian nesting dolls — outer runs first on the way in, last on the way out.

```
@BeforeSuite ← runs ONCE before everything @BeforeTest ← runs before each <test> block in XML @BeforeClass ← runs ONCE before first method in class @BeforeMethod ← runs before EVERY @Test method @Test ← your actual test @AfterMethod ← runs after EVERY @Test method @AfterClass ← runs ONCE after last method in class @AfterTest ← runs after each <test> block in XML @AfterSuite ← runs ONCE after everything
```

■ Pro Tip:

If a class has 3 @Test methods: BeforeClass runs 1 time, BeforeMethod runs 3 times, AfterMethod runs 3 times, AfterClass runs 1 time.

Q9. What is @BeforeSuite and when do you use it?

@BeforeSuite runs exactly once before any test in the entire suite starts. Use it for one-time global setup like connecting to a database, initializing a test report framework, or setting up test environment URLs.

```
@BeforeSuite public void globalSetup() { System.out.println("Connecting to DB..."); DBConnection.init("jdbc:mysql://testdb"); ExtentReport.init("test-results/report.html"); }
```

■ Real World: A team uses @BeforeSuite to start a Selenium Grid server before any browser test runs.

Q10. What is @BeforeTest and when do you use it?

@BeforeTest runs before any test method in a <test> block in testng.xml. If your XML has a 'SmokeTests' block and a 'RegressionTests' block, @BeforeTest runs once before each block. Use it for test-level data setup.

```
@BeforeTest public void setTestData() { testConfig = ConfigReader.load("qa-env.properties"); }
```

Q11. When should you use @BeforeClass vs @BeforeMethod?

@BeforeClass: Use when setup should happen ONCE for all tests in a class — e.g., launching a browser. Launching Chrome for every single test wastes time. @BeforeMethod: Use when each test needs a FRESH start — e.g., navigating to the homepage, or logging in fresh each time.

```
@BeforeClass // ONCE per class public void launchBrowser() { driver = new ChromeDriver(); } @BeforeMethod // BEFORE every @Test public void goToHomePage() { driver.get("https://myapp.com"); }
```

■ Pro Tip:

Rule of thumb: If it's expensive (browser launch, DB connect) → @BeforeClass. If it's a reset (clear cookies, navigate) → @BeforeMethod.

Q12. What is the @Factory annotation?

@Factory creates multiple instances of a test class dynamically at runtime. Unlike @DataProvider which repeats a method, @Factory repeats the whole class with different configurations.

```
public class LoginTest { private String browser; public LoginTest(String browser) { this.browser = browser; } @Test public void testLogin() { System.out.println("Testing on: " + browser); } } public class TestFactory { @Factory public Object[] createTests() { return new Object[] { new LoginTest("Chrome"), new LoginTest("Firefox"), new LoginTest("Edge") }; } } // Result: LoginTest runs 3 times, once per browser
```

Q13. What is the @Listeners annotation?

@Listeners attaches a listener class to a test class. A listener watches test events and reacts — like taking screenshots on failure or logging test start/end times. You place it above the class definition.

```
@Listeners(MyScreenshotListener.class) public class LoginTest { @Test public void testLogin() { // if this fails, MyScreenshotListener.onTestFailure() fires } }
```

Q14. Can configuration methods be run conditionally?

Yes. You can control when config methods run using: (1) enabled=false to completely skip a method, (2) groups to run config only for specific test groups, (3) dependsOnMethods to run setup only if a prerequisite test passed.

```
@BeforeMethod(enabled=false) public void skipThisSetup() { // this method will NEVER run }
@BeforeMethod(groups="smoke") public void smokeSetup() { // runs ONLY before tests in the 'smoke' group }
```

SECTION 3 — Priority, Groups, Dependencies & Timeouts

Q15. What is priority in TestNG and how does ordering work?

Priority tells TestNG which test to run first. Lower number = runs first. Default priority is 0. If two tests have the same priority, TestNG runs them alphabetically by method name.

```
@Test(priority=1) // runs 2nd public void searchProduct() { ... } @Test(priority=0) // runs 1st (lowest = first) public void login() { ... } @Test(priority=2) // runs 3rd public void addToCart() { ... } // Execution order: login → searchProduct → addToCart
```

■ Pro Tip:

Always use priority for tests that must follow a sequence — like login must come before any other feature test.

Q16. What happens if we don't set priority?

TestNG sorts the methods alphabetically by name. So addToCart() runs before login() simply because 'a' comes before 'l'. This can break tests that depend on order. Always set priority for flow-based tests.

```
@Test public void checkOut() { ... } // runs 1st (c) @Test public void login() { ... } // runs 2nd (l) @Test public void payment() { ... } // runs 3rd (p) // Problem: checkOut runs before login — test will FAIL!
```

Q17. What are TestNG groups and why are they useful?

Groups are labels (tags) you attach to tests. You can then choose to run only specific labels. This is how teams run 'smoke tests' (fast, critical tests) separately from 'regression tests' (full, thorough tests).

```
@Test(groups="smoke") public void loginTest() { ... } // tagged as smoke
@Test(groups={"smoke","regression"}) public void searchTest() { ... } // tagged as both
@Test(groups="regression") public void bulkOrderTest() { ... } // tagged as regression only // In testng.xml: include only "smoke" → runs loginTest + searchTest
```

■ Real World: After every code push, CI runs only 'smoke' group (2 min). Full 'regression' runs nightly (2 hours). Same codebase, different XML.

Q18. How do you include or exclude groups in testng.xml?

Use <groups> with <include> or <exclude> tags inside testng.xml. No code changes needed — just update the XML to control what runs.

```
<suite name="Suite"> <test name="SmokeRun"> <groups> <run> <include name="smoke"/> <!-- run smoke tests --> <exclude name="regression"/> <!-- skip regression --> </run> </groups> <classes> <class name="com.tests.LoginTest"/> </classes> </test> </suite>
```

Q19. What is the difference between dependency and priority?

Priority = ordering only. Even if a high-priority test FAILS, the next priority test still runs. Dependency = conditional execution. If the test you depend on FAILS, your test is automatically SKIPPED — not failed, just skipped.

```
// PRIORITY — order only, failure doesn't stop next test @Test(priority=1) public void login() { Assert.fail(); } // FAILS @Test(priority=2) public void search() { ... } // Still RUNS // DEPENDENCY — failure causes skip @Test public void login() { Assert.fail(); } // FAILS @Test(dependsOnMethods="login")
```

```
public void search() { ... } // SKIPPED
```

Q20. What are dependsOnMethods and dependsOnGroups?

dependsOnMethods: A test runs only if a specific method passed. dependsOnGroups: A test runs only if ALL tests in a group passed. Perfect for sequential workflows where each step builds on the previous one.

```
@Test(groups="auth") public void login() { ... } @Test(groups="auth") public void verifyDashboard() { ... } // search depends on ALL tests in "auth" group passing @Test(dependsOnGroups="auth") public void searchProduct() { ... } // checkout depends only on searchProduct method
@Test(dependsOnMethods="searchProduct") public void checkout() { ... }
```

■ Pro Tip:

Use dependsOnGroups for larger dependencies, dependsOnMethods for specific one-to-one dependencies.

Q21. What are timeOut, invocationCount, and threadPoolSize?

timeOut: If the test takes longer than X milliseconds, it FAILS automatically. invocationCount: Runs the same test N times (good for load/stress testing). threadPoolSize: When used with invocationCount, runs those N repetitions in parallel using multiple threads.

```
// Test must finish within 5 seconds or it fails @Test(timeOut=5000) public void pageLoadTest() {
driver.get("https://myapp.com"); // if page takes >5s, TestNG marks FAIL } // Run the same test 10 times
@Test(invocationCount=10) public void stressLoginTest() { ... } // Run 10 times using 3 parallel threads
@Test(invocationCount=10, threadPoolSize=3) public void parallelStressTest() { ... }
```

Q22. How do you run the same test case multiple times?

Use invocationCount in the @Test annotation. TestNG will execute that exact method the specified number of times. Useful for stability testing or verifying there are no intermittent failures.

```
@Test(invocationCount=5) public void testLoginRepeatedly() { driver.get("https://app.com/login"); // This method runs 5 times in a row Assert.assertEquals(driver.getTitle(), "Login Page"); }
```

Q23. How do you disable a test case in TestNG?

Set enabled=false in the @Test annotation. TestNG completely ignores that method during execution. The test stays in the code but never runs — great for temporarily disabling a broken test without deleting it.

```
@Test(enabled=false) public void brokenFeatureTest() { // this test is IGNORED during execution // use when feature is under development } @Test(enabled=true) // default, same as just @Test public void workingTest() { // this runs normally }
```

■ Pro Tip:

Never comment out tests. Use enabled=false so the test stays visible in code reviews and can be re-enabled easily.

Q24. How do you handle expected exceptions in TestNG?

Use the expectedExceptions attribute. If the test throws the specified exception, it PASSES. If it does NOT throw that exception, it FAILS. Perfect for testing error handling and validation logic.

```
@Test(expectedExceptions = {ArithmeticException.class}) public void testDivideByZero() { int result = 10 / 0; // throws ArithmeticException // TestNG sees the exception → marks test as PASSED }
@Test(expectedExceptions = {IllegalArgumentException.class}) public void testInvalidInput() {
MyService.process(null); // should throw IllegalArgumentException }
```

Q25. What is the successPercentage attribute?

When using invocationCount, successPercentage sets the minimum % of runs that must pass for the test to be considered overall PASSED. Useful for flaky tests that occasionally fail due to network or timing issues.

```
// Run 10 times, at least 80% (8 out of 10) must pass @Test(invocationCount=10, successPercentage=80)
public void flakyNetworkTest() { // Even if 2 out of 10 runs fail, overall test = PASSED }
```

Q26. What is @Test(singleThreaded=true)?

Even when parallel execution is ON for the suite, singleThreaded=true forces ALL methods of that specific class to run in a single thread, one after another. Use this to protect classes that are NOT thread-safe.

```
@Test(singleThreaded=true) public class OrderFlowTest { @Test public void addToCart() { ... } @Test public void checkout() { ... } @Test public void payment() { ... } // These 3 always run sequentially in one thread // even if parallel="methods" is set in testng.xml }
```

Q27. How do you skip a test in TestNG?

Throw a SkipException inside the test method. TestNG marks it as SKIPPED (not FAILED). Useful when a test should only run under certain conditions — like skipping a payment test if the payment gateway is down.

```
@Test public void paymentTest() { if (!PaymentGateway.isAvailable()) { throw new SkipException("Payment gateway is down, skipping test"); } // ... rest of payment test } // Result: Test marked as SKIPPED in report, not FAILED
```

SECTION 4 — Assertions — Hard vs Soft

Q28. What is the difference between hard and soft assertions?

Hard Assertion (Assert class): Stops the test IMMEDIATELY when one check fails. Like hitting a wall — nothing after it runs. Soft Assertion (SoftAssert class): Continues running even after a failure. Collects ALL failures and reports them together at the end when assertAll() is called.

```
// HARD ASSERTION — stops on first failure @Test public void hardAssertExample() { Assert.assertEquals(driver.getTitle(), "Home"); // FAILS here Assert.assertTrue(logo.isDisplayed()); // NEVER reaches this } // SOFT ASSERTION — continues after failure @Test public void softAssertExample() { SoftAssert soft = new SoftAssert(); soft.assertEquals(driver.getTitle(), "Home"); // fails but continues soft.assertTrue(logo.isDisplayed()); // also checked soft.assertEquals(driver.getCurrentUrl(), "/"); // also checked soft.assertAll(); // reports ALL 3 failures together }
```

■ Pro Tip:

Use hard assertions for critical preconditions (e.g., login must succeed). Use soft assertions for UI validation pages where you want to see all issues at once.

Q29. When should you prefer soft assertions in UI tests?

When you're validating MULTIPLE independent elements on the same page — like a dashboard with title, username, menu items, buttons, etc. If one element is wrong, you still want to know about all the others. Soft assertions give you the full picture in one run.

■ Example:

Validating a profile page: Check profile photo, username, email, phone, address all in one test. If email is wrong AND phone is wrong, you see BOTH failures — not just the first one.

```
SoftAssert soft = new SoftAssert(); soft.assertEquals(profilePhoto.isDisplayed(), true, "Photo missing"); soft.assertEquals(userNameField.getText(), "John", "Wrong username"); soft.assertEquals(emailField.getText(), "john@test.com", "Wrong email"); soft.assertEquals(phoneField.getText(), "+91-9999", "Wrong phone"); soft.assertAll(); // shows all failures at once
```

Q30. How do you ensure soft assertion failures are actually reported?

You MUST call assertAll() at the end of the test. This is mandatory. If you forget it, the test PASSES even if 10 assertions failed. The failures are silently ignored without assertAll().

```
SoftAssert soft = new SoftAssert(); soft.assertEquals(actual, expected); // failure collected // ... more assertions ... soft.assertAll(); // ← MANDATORY! Without this, test always PASSES // even if all assertions above failed!
```

■ Pro Tip:

Best practice: put assertAll() in a finally block or @AfterMethod to ensure it always runs.

Q31. What is a good assertion strategy to reduce flaky failures?

(1) Only assert things that are STABLE and important — not timestamps, random IDs, or dynamic content. (2) Always wait for elements properly (explicit waits) before asserting. (3) Keep assertions focused — one concept per assertion. (4) Use meaningful messages so failures explain themselves.

```
// BAD — flaky assertion on dynamic content Assert.assertEquals(lastLoginTime.getText(), "2 mins ago");
// changes! // GOOD — assert on stable, meaningful data Assert.assertEquals(welcomeMsg.getText(),
"Welcome, John!"); Assert.assertTrue(logoutBtn.isDisplayed(), "Logout button not visible"); // GOOD —
wait before asserting WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
wait.until(ExpectedConditions.visibilityOf(element)); Assert.assertTrue(element.isDisplayed());
```

Q32. What types of assertEquals are available in TestNG?

assertEquals is overloaded for all Java primitive types and their arrays: boolean, byte, int, short, long, char, float, double, String — each available as single value, array, and with an optional failure message parameter.

```
Assert.assertEquals(5, 5); // int Assert.assertEquals("Hello", "Hello"); // String
Assert.assertEquals(3.14, 3.14, 0.001); // double with delta Assert.assertEquals(true, true); // boolean
Assert.assertEquals(new int[]{1,2,3}, new int[]{1,2,3}); // array // With custom failure message:
Assert.assertEquals(actual, expected, "Login page title mismatch");
```

SECTION 5 — Parameterization & Data-Driven Testing

Q33. What is @Parameters in TestNG and where do values come from?

@Parameters passes values from testng.xml into your test method. You define the parameter in XML once, and any test that needs it gets it automatically. Perfect for environment-specific values like URLs, browser names, or usernames.

```
// testng.xml <suite name="Suite"> <parameter name="browser" value="chrome"/> <parameter name="env"
value="QA"/> <test name="Test1"> <classes><class name="LoginTest"/></classes> </test> </suite> //
LoginTest.java @Test @Parameters({"browser", "env"}) public void loginTest(String browser, String env) {
System.out.println("Running on: " + browser + " in " + env); // browser = "chrome", env = "QA" }
```

Q34. What is @DataProvider and how is it different from @Parameters?

@Parameters: Sends ONE set of values from XML. Good for config/environment setup. @DataProvider: Sends MULTIPLE sets of data to run the SAME test multiple times. Each row of data = one test run. Perfect for testing many input combinations.

```
// @DataProvider — runs loginTest 3 times with different users @DataProvider(name="loginData") public
Object[][] provideLoginData() { return new Object[][] { {"admin@test.com", "Admin@123"},
{"user@test.com", "User@456"}, {"guest@test.com", "Guest@789"} }; } @Test(dataProvider="loginData")
public void loginTest(String email, String password) { loginPage.login(email, password);
Assert.assertTrue(dashboardPage.isDisplayed()); } // Result: 3 separate test runs, each with different
credentials
```

■ Real World: An e-commerce site tests login with valid user, invalid password, locked account, and admin — all with one @Test method using @DataProvider.

Q35. How do you feed data from CSV/JSON/DB into a DataProvider?

Read the external data source in Java code, convert it into a 2D Object array, and return it from the @DataProvider method. TestNG handles the rest — it calls your test once per row automatically.

```
// Reading from CSV using Apache Commons CSV @DataProvider(name="csvData") public Object[][]
readFromCSV() throws Exception { List<Object[]> data = new ArrayList<>(); Reader reader =
Files.newBufferedReader(Paths.get("testdata.csv")); CSVParser parser = new CSVParser(reader,
CSVFormat.DEFAULT.withHeader()); for (CSVRecord record : parser) { data.add(new
Object[]{record.get("username"), record.get("password")}); } return data.toArray(new Object[0][]); }
@Test(dataProvider="csvData") public void loginWithCSVData(String username, String password) {
loginPage.login(username, password); }
```

Q36. How do you run DataProvider tests safely in parallel?

Enable parallel=true in the @DataProvider. But each thread must have its OWN WebDriver — use ThreadLocal to isolate drivers. Never use shared static variables for test data during parallel runs.

```
@DataProvider(name="users", parallel=true) public Object[][] getUsers() { return new Object[][]{{"user1"}, {"user2"}, {"user3"}}; } // ThreadLocal ensures each thread gets its own driver private ThreadLocal<WebDriver> driver = new ThreadLocal<>(); @BeforeMethod public void setup() { driver.set(new ChromeDriver()); } @Test(dataProvider="users") public void testUser(String username) { driver.get().get("https://app.com"); // each thread uses its own driver } @AfterMethod public void teardown() { driver.get().quit(); driver.remove(); }
```

Q37. How do you configure a test to use a DataProvider?

In the @Test annotation, set the dataProvider attribute to match the name defined in @DataProvider. If the DataProvider is in a different class, also set dataProviderClass attribute.

```
// DataProvider in same class @Test(dataProvider="myData") public void testMethod(String input) { ... }
// DataProvider in a different class @Test(dataProvider="myData", dataProviderClass=TestDataProvider.class) public void testMethod(String input) { ... }
```

Q38. How does TestNG support cross-browser testing?

Pass the browser name as a parameter from testng.xml. Your setup method reads this parameter and launches the corresponding driver. Define separate <test> blocks for each browser in the XML.

```
// testng.xml <test name="Chrome Tests"> <parameter name="browser" value="chrome"/> <classes><class name="SearchTest"/></classes> </test> <test name="Firefox Tests"> <parameter name="browser" value="firefox"/> <classes><class name="SearchTest"/></classes> </test> // Java setup method @BeforeMethod @Parameters("browser") public void setupBrowser(String browser) { if (browser.equals("chrome")) driver = new ChromeDriver(); if (browser.equals("firefox")) driver = new FirefoxDriver(); if (browser.equals("edge")) driver = new EdgeDriver(); }
```

SECTION 6 — Parallel Execution

Q39. What is parallel execution and how do you enable it?

Parallel execution runs multiple tests AT THE SAME TIME using multiple threads, instead of one after another. This dramatically cuts execution time for large test suites. Enable it in testng.xml by setting the parallel attribute and thread-count.

```
<!-- Run all test methods in parallel using 4 threads --> <suite name="MySuite" parallel="methods" thread-count="4"> <test name="AllTests"> <classes> <class name="com.tests.LoginTest"/> <class name="com.tests.SearchTest"/> <class name="com.tests.CartTest"/> </classes> </test> </suite>
```

■ Real World: Without parallel: 50 tests × 30 sec each = 25 minutes. With parallel (5 threads): ~5 minutes. Same tests, 5× faster.

Q40. What is the difference between parallel=methods, classes, and tests?

parallel=methods: Each @Test method runs in its own thread — fastest parallelism. parallel=classes: Each test CLASS runs in its own thread; methods inside one class still run sequentially. parallel=tests: Each <test> block in testng.xml runs in parallel — coarser control.

```
<!-- Methods level: most granular --> <suite parallel="methods" thread-count="5"> <!-- Classes level: class-by-class parallel --> <suite parallel="classes" thread-count="3"> <!-- Tests level: XML test-block parallel (good for cross-browser) --> <suite parallel="tests" thread-count="2"> <test name="ChromeTests"> ... </test> <test name="FirefoxTests"> ... </test> <!-- Both test blocks run simultaneously --> </suite>
```

■ **Pro Tip:**

For cross-browser parallel execution, use parallel='tests'. For maximum speed within one browser, use parallel='methods'.

Q41. What is thread-count and how do you choose a safe value?

thread-count is the number of tests that run simultaneously. More threads = faster, but more CPU/RAM/browser usage. A safe starting point is 3–5. Test with small values and increase until you find the sweet spot where tests are stable and fast.

■ **Example:**

If your machine has 8 GB RAM and each Chrome browser uses ~300 MB, you can safely run about 10 parallel browsers before running out of memory.

■ **Pro Tip:**

Start with thread-count=3. If tests are stable, try 5. If still stable, try 8. Stop when you see test failures due to resource contention.

Q42. Why do Selenium tests fail in parallel and how do you fix it?

The most common cause: multiple threads share ONE WebDriver object. Thread A clicks a button while Thread B navigates away — chaos. Fix: use ThreadLocal<WebDriver> to give EACH thread its own private driver instance.

```
// WRONG — shared driver causes parallel test failures public class BaseTest { public static WebDriver driver; // shared across ALL threads! } // CORRECT — each thread gets its own driver public class BaseTest { private static ThreadLocal<WebDriver> driver = new ThreadLocal<>(); public WebDriver getDriver() { return driver.get(); } @BeforeMethod public void setup() { driver.set(new ChromeDriver()); // new driver for THIS thread } @AfterMethod public void teardown() { driver.get().quit(); driver.remove(); // clean up thread-local } }
```

Q43. How do you handle shared test data in parallel runs?

Avoid static or shared mutable data. Each test thread should work with its own copy of test data. Use ThreadLocal for any thread-specific data, and prefer immutable data objects that multiple threads can safely read.

```
// WRONG — static data gets overwritten by parallel threads public class TestData { public static String userName; // Thread A sets "Alice", Thread B sets "Bob" → conflict! } // CORRECT — ThreadLocal data per thread public class TestData { private static ThreadLocal<String> userName = new ThreadLocal<>(); public static void setUser(String u) { userName.set(u); } public static String getUser() { return userName.get(); } }
```

SECTION 7 — testng.xml Configuration

Q44. What is testng.xml and what problems does it solve?

testng.xml is the control center of TestNG. It lets you decide WHAT runs, WHEN it runs, HOW MANY threads to use, and WHICH groups to include — all without touching Java code. One codebase, multiple XMLs for different scenarios.

```
<?xml version="1.0" encoding="UTF-8"?> <!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd"> <suite name="E-Commerce Suite" parallel="tests" thread-count="3"> <parameter name="baseUrl" value="https://qa.myapp.com"/> <test name="Smoke Tests"> <groups><run><include name="smoke"/></run></groups> <classes> <class name="com.tests.LoginTest"/> <class name="com.tests.HomeTest"/> </classes> </test> </suite>
```

■ Real World: One team maintains 3 XML files: smoke.xml (runs in 2 min after every push), regression.xml (runs nightly), and sanity.xml (runs before release). Same test code, different configs.

Q45. How do you organize suites for smoke/regression/sanity?

Option 1: Separate XML files for each type (smoke.xml, regression.xml). Option 2: One XML with multiple <test> blocks. Groups are the cleanest approach — tag your tests and control execution purely from XML.

```
<!-- smoke.xml --> <suite name="Smoke"> <test name="Quick Smoke Check"> <groups><run><include name="smoke"/></run></groups> <classes><class name="com.tests.AllTests"/></classes> </test> </suite> <!-- regression.xml --> <suite name="Regression"> <test name="Full Regression"> <groups><run><include name="regression"/></run></groups> <classes><class name="com.tests.AllTests"/></classes> </test> </suite>
```

Q46. How do you run only specific classes or methods from testng.xml?

Under <classes>, list exact class names. For specific methods within a class, use the <methods> tag with <include>. This gives you surgical control over exactly what runs without changing any code.

```
<classes> <class name="com.tests.LoginTest"> <methods> <include name="loginWithValidCredentials"/> <include name="loginWithInvalidPassword"/> <!-- only these 2 methods run from LoginTest --> </methods> </class> <class name="com.tests.SearchTest"/> <!-- all methods run --> </classes>
```

Q47. How do you exclude specific tests or methods?

Use <exclude> tags either inside <methods> for specific methods, or inside <groups> <run> for entire groups. This is perfect for temporarily skipping broken tests without deleting or modifying Java code.

```
<classes> <class name="com.tests.PaymentTest"> <methods> <exclude name="refundTest"/> <!-- skip only this method --> </methods> </class> </classes> <!-- OR exclude an entire group --> <groups> <run> <include name="regression"/> <exclude name="broken"/> <!-- skip all tests tagged "broken" --> </run> </groups>
```

Q48. How do you run failed test cases in TestNG?

TestNG automatically creates a testng-failed.xml file in the test-output folder after any test run that has failures. Just right-click that file and run it as a TestNG Suite to re-execute only the failed tests. No manual work needed.

```
// After a test run, check: // project/test-output/testng-failed.xml // This file is auto-generated and contains only the failed tests. // Run it: Right-click testng-failed.xml → Run As → TestNG Suite // In Maven CI: // mvn test -Dsurefire.suiteXmlFiles=test-output/testng-failed.xml
```

■ **Pro Tip:**

In Jenkins, you can configure the pipeline to automatically re-run using testng-failed.xml when the first run has failures.

Q49. How do you create a testng.xml file in Eclipse/IntelliJ?

In Eclipse: Right-click the package → TestNG → Convert to TestNG. This auto-generates testng.xml. In IntelliJ: Run the test class first, then export the run configuration as testng.xml. You can also write it manually.

```
<!-- Minimal testng.xml you can write manually --> <?xml version="1.0" encoding="UTF-8"?> <!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd"> <suite name="MySuite"> <test name="MyTest"> <classes> <class name="com.myapp.tests.LoginTest"/> </classes> </test> </suite>
```

SECTION 8 — Listeners, Retry & Custom Reporting

Q50. What are TestNG listeners and why do teams use them?

Listeners are Java classes that 'listen' to TestNG events — test start, pass, fail, skip, suite start, suite end. When an event fires, your listener code runs automatically. Teams use them to take screenshots on failure, log test results, send Slack/email alerts, or update Jira tickets — without touching test methods.

```
public class MyListener implements ITestListener { @Override public void onTestFailure(ITestResult result) { // Automatically runs when ANY test fails System.out.println("FAILED: " + result.getName()); takeScreenshot(result.getName()); sendSlackAlert("Test failed: " + result.getName()); } @Override public void onTestSuccess(ITestResult result) { System.out.println("PASSED: " + result.getName()); } }
```

■ Real World: A team's listener automatically creates a Jira bug when a test fails in production regression — zero manual effort.

Q51. What is ITestListener and what events does it capture?

ITestListener is the most commonly used listener interface. It captures 8 key events for each test method in the suite.

```
public class MyTestListener implements ITestListener { void onTestStart(ITestResult r) // test is about to run void onTestSuccess(ITestResult r) // test passed void onTestFailure(ITestResult r) // test failed void onTestSkipped(ITestResult r) // test was skipped void onTestFailedButWithinSuccessPercentage(ITestResult r) void onTestFailedWithTimeout(ITestResult r) void onStart(ITestContext ctx) // before any test in a <test> block void onFinish(ITestContext ctx) // after all tests in a <test> block }
```

Q52. What is ISuiteListener and when do you use it?

ISuiteListener fires before and after the ENTIRE suite (all tests combined). Use it for suite-level actions: initializing a report framework before any test runs, or generating a final summary report after everything finishes.

```
public class SuiteListener implements ISuiteListener { @Override public void onStart(ISuite suite) { // Before ANY test runs ExtentReports.setup("test-results/report.html"); System.out.println("Suite starting: " + suite.getName()); } @Override public void onFinish(ISuite suite) { // After ALL tests finish ExtentReports.flush(); // write final report EmailUtil.sendReport("team@company.com", "report.html"); } }
```

Q53. How do you capture screenshots on failure using listeners?

Implement ITestListener and in the onTestFailure method, use Selenium's TakesScreenshot interface to capture and save a screenshot with the test name. Attach the listener to your class using @Listeners.

```
public class ScreenshotListener implements ITestListener { @Override public void onTestFailure(ITestResult result) { // Get the driver from the test class Object testClass = result.getInstance(); WebDriver driver = ((BaseTest) testClass).getDriver(); // Take screenshot TakesScreenshot ts = (TakesScreenshot) driver; File src = ts.getScreenshotAs(OutputType.FILE); // Save with test name + timestamp String path = "screenshots/" + result.getName() + "_" + System.currentTimeMillis() + ".png"; try { FileUtils.copyFile(src, new File(path)); } catch (IOException e) { e.printStackTrace(); } } } // Apply to test class: @Listeners(ScreenshotListener.class) public class LoginTest extends BaseTest { ... }
```

Q54. What is a RetryAnalyzer and when should you avoid it?

RetryAnalyzer automatically re-runs a failed test a set number of times before marking it as truly failed. Useful for flaky tests caused by network hiccups or timing issues. AVOID retries when the failure is due to a real application bug — retries will hide the defect and give false confidence.

```
// Step 1: Create the RetryAnalyzer public class RetryAnalyzer implements IRetryAnalyzer { private int count = 0; private static final int MAX_RETRIES = 2; // retry up to 2 times @Override public boolean retry(ITestResult result) { if (count < MAX_RETRIES) { count++; return true; // retry this test } return false; // stop retrying } } // Step 2: Attach to test @Test(retryAnalyzer = RetryAnalyzer.class) public void flakyNetworkTest() { ... } // If fails: retries once → retries twice → marked as FAILED
```

■ Pro Tip:

Use retries ONLY for known infrastructure flakiness (network timeouts, CI server load). Never use them to mask actual bugs.

Q55. What is the IReporter interface?

IReporter allows you to generate completely custom test reports after the entire suite finishes. Implement generateReport() to create any output format — HTML, PDF, CSV, JSON — with exactly the data and design you want.

```
public class CustomReporter implements IReporter { @Override public void generateReport(List<XmlSuite> xmlSuites, List<ISuite> suites, String outputDirectory) { for (ISuite suite : suites) { Map<String, ISuiteResult> results = suite.getResults(); for (String test : results.keySet()) { ITestContext ctx = results.get(test).getTestContext(); System.out.println("PASSED: " + ctx.getPassedTests().size()); System.out.println("FAILED: " + ctx.getFailedTests().size()); } } // Write to custom HTML/JSON/Excel file here } }
```

Q56. What is the ITestResult interface?

ITestResult is an object passed to listener methods that contains complete information about a test execution — its name, status, start time, end time, thrown exception, and the test instance. You use it inside listener methods to extract test details.

```
public void onTestFailure(ITestResult result) { result.getName() // "loginTest" result.getStatus() // ITestResult.FAILURE = 2 result.getThrowable() // the exception that caused failure result.getStartMillis() // when test started result.getEndMillis() // when test ended result.getInstance() // the test class object result.getMethod() // the TestNG method object result.getParameters() // data from @DataProvider }
```

Q57. What is IInvokedMethodListener?

IInvokedMethodListener fires before and after EVERY method invocation — including @BeforeMethod, @AfterMethod, and @Test methods. More granular than ITestListener. Use it for precise timing, logging each method call, or applying pre/post conditions to every invocation.

```
public class MethodLogger implements IInvokedMethodListener { @Override public void beforeInvocation(IInvokedMethod method, ITestResult result) { System.out.println("Starting: " + method.getTestMethod().getMethodName()); } @Override public void afterInvocation(IInvokedMethod method, ITestResult result) { long duration = result.getEndMillis() - result.getStartMillis(); System.out.println("Done: " + method.getTestMethod().getMethodName() + " in " + duration + "ms"); } }
```

Q58. How do you generate and customize reports in TestNG?

TestNG auto-generates basic HTML and XML reports in the test-output folder. For professional reports, integrate Extent Reports or Allure through listeners. These tools create rich, interactive reports with screenshots, logs, trends, and charts.

```
// Extent Reports integration via listener public class ExtentListener implements ITestListener {
ExtentReports extent = new ExtentReports(); ExtentTest test; public void onTestStart(ITestResult result)
{ test = extent.createTest(result.getName()); } public void onTestSuccess(ITestResult result) {
test.pass("Test Passed"); } public void onTestFailure(ITestResult result) {
test.fail(result.getThrowable()); test.addScreenCaptureFromPath(takeScreenshot(result.getName())); }
public void onFinish(ITestContext ctx) { extent.flush(); // write report to disk } }
```

■ **Pro Tip:**

Allure Reports provide the most beautiful and interactive reports with history trends, categories, and environment info.

Q59. How do you log messages in TestNG reports?

Use Reporter.log() to add custom messages that appear directly in TestNG's HTML report alongside each test result. This is better than System.out.println() because the messages are tied to the specific test in the report.

```
@Test public void loginTest() { Reporter.log("Navigating to login page...");
driver.get("https://myapp.com/login"); Reporter.log("Entering credentials for: admin@test.com");
loginPage.enterEmail("admin@test.com"); loginPage.enterPassword("Admin@123"); loginPage.clickLogin();
Reporter.log("Verifying dashboard is displayed"); Assert.assertTrue(dashboard.isDisplayed()); // All
these messages appear in the HTML report under this test }
```

Q60. What is an annotation transformer in TestNG?

Annotation transformers use IAnnotationTransformer to modify @Test annotation values at RUNTIME — before tests execute. You can dynamically enable/disable tests, change priorities, or assign retry analyzers based on external data (like a database or config file).

```
public class MyTransformer implements IAnnotationTransformer { @Override public void
transform(ITestAnnotation annotation, Class testClass, Constructor testConstructor, Method testMethod) {
// Disable all tests tagged "flaky" at runtime if (testMethod.isAnnotationPresent(Flaky.class)) {
annotation.setEnabled(false); } // Assign retry to all tests automatically
annotation.setRetryAnalyzer(RetryAnalyzer.class); } }
```

■ Real World: A team reads a 'disabled tests' list from their CI config at runtime and uses a transformer to skip those tests without modifying any code.

SECTION 9 — Build & CI (Maven + Jenkins)

Q61. How do you run TestNG tests using Maven?

Maven uses the maven-surefire-plugin to find and run TestNG tests. Add the plugin to pom.xml and specify your testng.xml file. Then run 'mvn test' from the terminal — no IDE needed. This is how tests run in CI pipelines.

```
<!-- pom.xml --> <build> <plugins> <plugin> <groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId> <version>3.0.0</version> <configuration> <suiteXmlFiles>
<suiteXmlFile>testng.xml</suiteXmlFile> </suiteXmlFiles> </configuration> </plugin> </plugins> </build>
<!-- Run from terminal: --> <!-- mvn test (default testng.xml) --> <!-- mvn test -Dsuite=smoke.xml
(custom suite file) -->
```

Q62. How do you pass suite files, groups, or parameters from command line?

Use Maven system properties with -D flag. Your testng.xml or framework code reads these properties at runtime. This makes the same codebase work for different environments, browsers, and test types without any code changes.

```
# Run specific suite file mvn test -DsuiteXmlFile=smoke.xml # Pass browser and environment mvn test
-Dbrowser=firefox -Denv=UAT # In Java, read the system property: String browser =
System.getProperty("browser", "chrome"); // default: chrome String env = System.getProperty("env", "QA");
```

■ Real World: Jenkins passes -Denv=PROD -Dbrowser=chrome -DsuiteXmlFile=regression.xml. The same code runs in QA, UAT, and PROD just by changing Jenkins parameters.

Q63. How do you publish TestNG reports in Jenkins?

After tests run, TestNG generates reports in the test-output folder. Configure Jenkins to archive this folder as build artifacts. Use the 'Publish TestNG Results' Jenkins plugin to show pass/fail charts directly on the Jenkins job dashboard.

```
// In Jenkins Pipeline (Jenkinsfile): pipeline { stages { stage('Test') { steps { sh 'mvn test' } post { always { // Archive TestNG reports testNG reportFilenamePattern: 'test-output/testng-results.xml' archiveArtifacts artifacts: 'test-output/**/*', allowEmptyArchive: true } } } } }
```

Q64. How do you manage environment-specific configurations in CI?

Maintain separate properties files for each environment (qa.properties, uat.properties, prod.properties). Jenkins passes the environment name as a parameter. A config reader class loads the right file at runtime. Zero code changes for environment switching.

```
// ConfigReader.java public class ConfigReader { private static Properties props = new Properties(); static { String env = System.getProperty("env", "qa"); // default: qa String file = "src/test/resources/" + env + ".properties"; try { props.load(new FileInputStream(file)); } catch (Exception e) { e.printStackTrace(); } } public static String get(String key) { return props.getProperty(key); } } // Usage in test: String baseUrl = ConfigReader.get("base.url"); // reads from qa/uat/prod.properties
```

SECTION 10 — Selenium + TestNG Framework Integration

Q65. Why is TestNG commonly used with Selenium automation?

Selenium only automates browser actions. It has no test management capabilities. TestNG adds everything Selenium is missing: execution order, grouping, parallel runs, data-driven testing, and detailed reports. Together they form a complete automation framework.

■ Example:

Selenium: driver.click(loginBtn) — that's all it does. TestNG: run this in Chrome AND Firefox in parallel, 3 times with different users, take screenshot if it fails, include in smoke group, report pass/fail with timing.

Q66. How do you structure a TestNG + Selenium automation framework?

Follow the Page Object Model (POM) pattern with a layered structure: BaseTest for setup/teardown, Pages for UI interactions, Tests for test logic, Utils for helpers, and Config for environment data.

```
project/ ■■■ src/test/java/ ■ ■■■ base/ ■ ■ ■■■ BaseTest.java ← driver init, @BeforeMethod/@AfterMethod ■ ■■■ pages/ ■ ■ ■■■ LoginPage.java ← locators + page actions ■ ■ ■■■ HomePage.java ■ ■■■ tests/ ■ ■ ■■■ LoginTest.java ← @Test methods only ■ ■ ■■■ SearchTest.java ■ ■■■ utils/ ■ ■■■ ConfigReader.java ← reads properties files ■ ■■■ WaitHelper.java ← explicit wait methods ■ ■■■ ScreenshotUtil.java ← screenshot capture ■■■ src/test/resources/ ■ ■■■ qa.properties ■ ■■■ testdata/ ■ ■■■ loginData.csv ■■■ testng.xml
```

■ Pro Tip:

Test classes should ONLY contain @Test methods. No driver code, no waits, no locators. All of that goes in Pages and Utils.

Q67. How do you manage the WebDriver lifecycle using TestNG annotations?

Initialize driver in @BeforeClass (once per class) or @BeforeMethod (before each test). Use driver in @Test methods. Quit driver in @AfterClass or @AfterMethod. For parallel tests, always use ThreadLocal.

```
public class BaseTest { private ThreadLocal<WebDriver> driver = new ThreadLocal<>(); public WebDriver getDriver() { return driver.get(); } @BeforeMethod public void setup() { WebDriver d = new ChromeDriver(); d.manage().window().maximize(); d.manage().timeouts().implicitlyWait(Duration.ofSeconds(10)); driver.set(d); } @AfterMethod public void teardown() { if (driver.get() != null) { driver.get().quit(); driver.remove(); } } } public class LoginTest extends BaseTest { @Test public void verifyLogin() { getDriver().get("https://myapp.com"); // use getDriver() everywhere — thread-safe } }
```

Q68. How do you implement tagging (smoke/regression) using groups?

Tag @Test methods with groups that reflect their test type. Run specific groups from testng.xml or Maven. This is the standard industry approach for managing large test suites.

```
@Test(groups={"smoke", "regression"}, priority=1) public void loginTest() { ... } // runs in both smoke + regression @Test(groups={"regression"}, priority=2) public void bulkUploadTest() { ... } // only in regression @Test(groups={"smoke"}, priority=1) public void homePageTest() { ... } // only in smoke // testng.xml for smoke run: // <include name="smoke"/> → runs loginTest + homePageTest only // testng.xml for regression run: // <include name="regression"/> → runs loginTest + bulkUploadTest
```

Q69. How do you do parallel cross-browser execution using TestNG?

Define separate <test> blocks per browser in testng.xml with parallel='tests'. Each block passes a different browser parameter. The setup method reads the parameter and launches the right driver. All browsers run simultaneously.

```
<!-- testng.xml – parallel cross-browser --> <suite name="CrossBrowser" parallel="tests" thread-count="3"> <test name="Chrome"> <parameter name="browser" value="chrome"/> <classes><class name="com.tests.LoginTest"/></classes> </test> <test name="Firefox"> <parameter name="browser" value="firefox"/> <classes><class name="com.tests.LoginTest"/></classes> </test> <test name="Edge"> <parameter name="browser" value="edge"/> <classes><class name="com.tests.LoginTest"/></classes> </test> </suite> // LoginTest runs on Chrome, Firefox, and Edge simultaneously!
```

Q70. What is the default order of execution when a class has 2 @Test, 1 @BeforeMethod, and 1 @AfterMethod?

@BeforeMethod wraps EVERY @Test. So with 2 tests, the pattern repeats twice: BeforeMethod runs → Test 1 runs → AfterMethod runs → BeforeMethod runs → Test 2 runs → AfterMethod runs.

```
public class OrderDemo { @BeforeMethod public void setup() { System.out.println("BEFORE"); } @Test public void testA() { System.out.println("TEST A"); } @Test public void testB() { System.out.println("TEST B"); } @AfterMethod public void teardown() { System.out.println("AFTER"); } } // OUTPUT: // BEFORE // TEST A // AFTER // BEFORE // TEST B // AFTER
```

■ Pro Tip:

This is why @BeforeMethod is perfect for resetting state — it gives each test a clean, fresh starting point.

Q71. What is the default priority if not set in TestNG?

Default priority is 0. Tests with no priority set all get priority 0 and run in alphabetical order of method name. This can lead to unexpected execution order in flow-based tests. Always set priority explicitly for any test that depends on order.

```
@Test public void checkout() { ... } // priority 0, runs 1st (c) @Test public void login() { ... } // priority 0, runs 2nd (l) @Test public void payment() { ... } // priority 0, runs 3rd (p) // PROBLEM: checkout runs before login – this test will fail! // FIX: @Test(priority=1) public void login() { ... } @Test(priority=2) public void checkout() { ... } @Test(priority=3) public void payment() { ... }
```

Q72. What is the TestNG @Test(singleThreaded=true) annotation used for?

Forces all methods in a class to run in a single thread sequentially, even when the suite has parallel execution enabled. Use this for classes where test methods share state or a single browser session that must not be interrupted by other threads.

```
// This class has a single browser session flowing through all tests @Test(singleThreaded=true) public class EndToEndOrderFlow extends BaseTest { @Test(priority=1) public void login() { ... } @Test(priority=2) public void searchProduct(){ ... } @Test(priority=3) public void addToCart() { ... } @Test(priority=4) public void checkout() { ... } @Test(priority=5) public void payment() { ... } // Even with parallel="methods" in suite, these 5 always run // in sequence in one thread – they share browser state }
```

Q73. How do you perform parallel cross-browser testing vs parallel testing?

Parallel Testing: Same test runs on multiple browsers AT THE SAME TIME. All browsers run simultaneously using thread-count. Cross-Browser Testing: Same test runs on multiple browsers ONE AT A TIME. No parallel config — tests run sequentially.

```
<!-- PARALLEL testing — Chrome + Firefox run simultaneously --> <suite name="Parallel" parallel="tests" thread-count="2"> <test name="Chrome"><parameter name="browser" value="chrome"/>...</test> <test name="Firefox"><parameter name="browser" value="firefox"/>...</test> </suite> <!-- CROSS-BROWSER testing — Chrome first, then Firefox --> <suite name="CrossBrowser"> <!-- no parallel attribute --> <test name="Chrome"><parameter name="browser" value="chrome"/>...</test> <test name="Firefox"><parameter name="browser" value="firefox"/>...</test> </suite>
```

■ **Pro Tip:**

For CI pipelines: use parallel testing for speed. For debugging: use cross-browser (sequential) to see exactly what happens on each browser.

Q74. How do you create a dependent test group?

Use dependsOnGroups in the @Test annotation. The test only runs after ALL tests in the specified group have passed. If any test in the group fails, this test is skipped — not failed, skipped.

```
@Test(groups="authentication") public void login() { ... } // group: authentication @Test(groups="authentication") public void verifySession() { ... } // group: authentication // This test runs ONLY after both authentication tests pass @Test(dependsOnGroups="authentication") public void placeOrder() { ... } // If login() fails → verifySession() may still run // → placeOrder() is SKIPPED (dependency not met)
```

Q75. How does TestNG handle the complete E2E flow using dependsOnMethods?

Chain tests using dependsOnMethods to enforce a strict flow. If any link in the chain fails, all subsequent tests are automatically skipped — which is exactly the right behavior for end-to-end flows.

```
@Test public void login() { // Step 1: Login loginPage.login("user@test.com", "Pass@123"); Assert.assertTrue(dashboard.isDisplayed()); } @Test(dependsOnMethods="login") public void searchProduct() { // Step 2: Only runs if login() passed searchPage.search("Laptop"); Assert.assertTrue(results.isDisplayed()); } @Test(dependsOnMethods="searchProduct") public void addToCart() { // Step 3: Only runs if searchProduct() passed resultsPage.clickFirstResult(); productPage.addToCart(); } // If login() fails: searchProduct = SKIPPED, addToCart = SKIPPED
```

■ Real World: A complete checkout flow: login → search → addToCart → checkout → payment → orderConfirmation. 6 dependent steps — if login fails, nothing else runs.